OmniFlow: Adaptive Memory Reuse for Mobile GUI Agents

Abstract

Mobile GUI agents typically execute tasks through an observe-reason-act loop, repeatedly invoking large vision-language models to determine each action. While effective for novel tasks, this execution model incurs substantial latency and computational overhead when users repeatedly perform similar routines. Reusing prior successful executions offers a promising way to bypass repeated reasoning, yet existing replay and memory-based approaches remain fragile in dynamic mobile environments. We present OmniFlow, an adaptive replay system that enables mobile GUI agents to reuse historical executions whenever they remain applicable. OmniFlow organizes successful executions into fine-grained, composable functions and uses lightweight GUI-state representations to rapidly retrieve reusable operations from intermediate workflow states. It further introduces OmniTransfer, a lightweight context-aware action re-grounding model that transfers recorded actions to corresponding elements in the live GUI state. We implement OmniFlow as a plug-in cache layer for existing mobile GUI agents. Across three dynamic execution environments, OmniFlow outperforms other memory-based GUI agents in task success rate while reducing token consumption by 50.8%. Results on micro-benchmarks demonstrate that OmniFlow further achieves 93.75% top-1 GUI-state retrieval accuracy and 82.74% top-1 action re-grounding accuracy, with low runtime overhead on physical mobile devices.

1 Introduction

Mobile GUI agents powered by vision-language models (VLMs) have demonstrated impressive capabilities in open-ended task execution. However, when deployed in the real world, users rarely utilize these agents solely for open-ended exploration. Instead, users expect them to act as intelligent assistants that accelerate daily, repeated routines–such as checking schedules, filling out common forms, or navigating familiar app workflows–fast and reliably. Currently, mobile GUI agents address these tasks using a standard observe-reason-act loop [? ? ? ?], as shown in Figure 1(a): observing the current screen, invoking a massive model to decide the next action, executing the operation, and repeating until completion. While this design is highly effective for novel tasks with the support of performant VLMs, applying step-wise VLM reasoning to recurring routines forces every user and every repetition to solve nearly the same interaction sequence from scratch. This repeated model invocation introduces severe computation, memory, energy, and thermal overheads on edge devices. This reality motivates a new

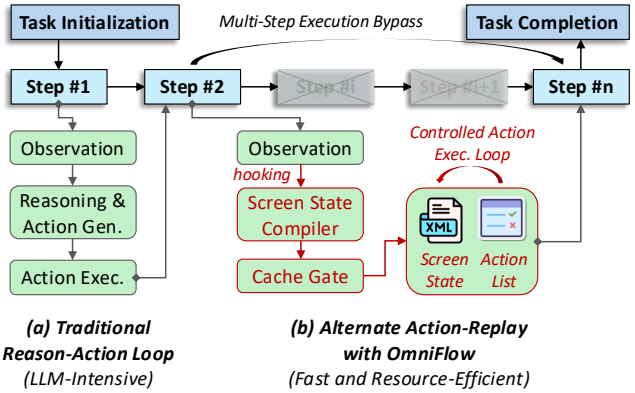


Figure 1. Workflow comparison between (a) a traditional LLM-/VLM-intensive reason-action loop and (b) a fast action-replay mechanism to accelerate task execution. By hooking observations and querying the memory, the system recognizes familiar screen states and adaptively replays a cached list of atomic actions.

serving target for mobile GUI agents: *successful execution histories of daily routines should be efficiently reused to bypass expensive model invocations whenever possible.*

Existing systems have attempted to address this serving target through various approaches, as summarized in Table 1. (1) Per-step acceleration methods [? ? ? ?] reduce the latency of individual model calls but fail to break the expensive observe-reason-act loop. (2) Traditional RPA-style replay and script-based tools [? ? ? ?] eliminate model calls by binding execution to fixed coordinates or fixed UI selectors, yet they are highly brittle to GUI dynamics. For example, a button may shift after a screen-size change, or its label may change after an app update, causing a previously recorded action to target the wrong element or fail entirely. (3) Memory-based agents [? ? ?], as Figure 1(b) shows, retrieve prior task execution history but still require online VLM reasoning to interpret state mismatches and adapt the retrieved actions to the current GUI state. This leaves such approaches too heavy for seamless on-device execution.

Crucially, these prior studies fall short because they cannot accommodate the complex realities of real-world mobile environments, which are driven by two fundamental dynamics. First, **workflow-level dynamics** mean that live tasks frequently diverge from recorded trajectories due to unpredictable interruptions such as login screens, pop-up ads, loading delays, or alternative app entry points. A rigid script fails entirely once the recorded path is disrupted. Second, **GUI-state-level dynamics** mean that even when the

overall workflow remains aligned, the concrete action target may shift due to, e.g., different device resolutions, font scales, or pop-ups. Thus, the memorized action may remain semantically correct while its recorded coordinate or UI selector no longer identifies the intended target. Together, these two forms of dynamics require execution reuse to adapt both which step to execute and how that step is grounded in the current GUI.

To this end, we propose OmniFlow, an adaptive on-device replay system for mobile GUI agents. OmniFlow is designed to replay execution history whenever possible, while falling back to VLM reasoning when memory cannot be reliably applied. This system is built around two core components: (1) the Omni representation layer, which decomposes past executions into a fine-grained GUI transition graph of reusable functions and enables fast online state matching through a lightweight, device-friendly embedding design; and (2) the Flow execution layer, which rapidly locates applicable memory from the graph and transfers recalled actions to the live GUI state through adaptive action re-grounding. These two components are specifically designed to solve the aforementioned dynamics through two core technical innovations.

To handle workflow-level dynamics, OmniFlow enables history replay to alternate with VLM reasoning and resume as soon as execution returns to a memorized GUI state. It incorporates two mechanisms to achieve this flexibility. (1) A fine-grained GUI transition graph that organizes historical executions into reusable Functions. The graph and its Functions are constructed offline through VLM-based analysis. Each GUI state records the functions available from that state, and multiple functions can be composed to accomplish higher-level tasks. This design allows replay to stop and resume at intermediate states without requiring the entire recorded trajectory to remain valid. (2) A lightweight on-device state-matching mechanism that rapidly relocates the live GUI within the graph. OmniFlow represents each GUI state with a feature-engineered page vector built from its view hierarchy, avoiding computation-heavy visual embeddings. This enables fast on-device similarity matching, allowing OmniFlow to quickly retrieve an applicable Function and resume replay after temporary VLM reasoning.

To tackle GUI-state-level dynamics, OmniFlow introduces a lightweight context-aware action re-grounding mechanism in the Flow execution layer. It first encodes the recorded target and live GUI elements using complementary text, visual, widget, and geometric features. It then calibrates each element with its local structural and spatial context, allowing visually or semantically similar elements to be distinguished by their surroundings. Finally, a lightweight element matcher ranks candidate targets on the live GUI state and executes only high-confidence matches. This design adapts memorized actions to changing layouts and GUI attributes without repeatedly invoking an expensive VLM for action transfer.

Table 1. Execution acceleration methods for mobile GUI agents. The columns capture whether a method (1) requires prior exploration on GUI states, (2) utilizes a cached memory to avoid mandatory LLM/VLM invocation at every step, and (3) transfers its memory to dynamic and heterogeneous execution environments.

Method	W/ App Exploration	W/ Model Invocation	Cross-Env Transferability
RPA/script replay [? ? ? ?]	✓	✗	✗
Memory-based [? ? ?]	✓	✓	✗
Program-guided [? ? ? ?]	✓	✗	✗
Per-step acceleration [? ? ? ?]	✗	✓	✗
OmniFlow	✗	✗	✓

We implement OmniFlow as a plug-in cache layer for existing mobile GUI agents. We evaluate OmniFlow on AndroidWorld [**?**] against three representative memory-based agents: MobileGPT [**?**], AutoDroid [**?**], and AppAgent [**?**]. We construct three dynamic execution environments by varying Android emulator configurations and task initialization parameters. Across these environments, OmniFlow achieves a 53.2% task success rate, outperforming prior memory-based mobile GUI agents. OmniFlow also reduces token consumption by 50.8% on successful executions. Micro-benchmarks further demonstrate 93.75% R@1 for GUI-state retrieval and 82.74% top-1 accuracy for action re-grounding. Experiments on physical mobile devices further confirm the low runtime overhead of OmniFlow.

This paper makes the following contributions.

- We characterize repetitive GUI automation as a key workload for on-device GUI agents and identify that existing memory designs are vulnerable to workflow- and GUI-state dynamics.
- We present OmniFlow, an adaptive on-device replay system for robust memory reuse in dynamic environments.
- We implement and evaluate OmniFlow, demonstrating higher execution success than existing memory-based agents under dynamic environments.

2 Background and Motivation

2.1 High Execution Costs of GUI Agents

Mobile GUI agents typically operate in an observe–reason–act loop [**? ? ? ? ?**]. At each step, the agent perceives the current screen, invokes an LLM or VLM to determine the next GUI action, and executes the action in the environment. Although this workflow enables flexible reasoning over open-ended tasks, it introduces substantial runtime overhead because model inference is repeatedly triggered during long interaction sequences. The cost accumulates with each additional step, leading to increased latency and computation during task execution [**? ? ? ? ?**]. Prior work reports that existing mobile agents often require more than 10 seconds per step,

1–2 minutes, and about $1 monetary cost per task on average [? ?]. More broadly, planning and reflection alone can account for 75%–94% of end-to-end latency in GUI agents [?]. These costs become more pronounced when the step-wise model inference runs directly on mobile devices with strict memory, power, and thermal constraints [? ? ?].

However, real-world mobile usage rarely consists of purely open-ended tasks. In practice, users repeatedly perform similar routines, such as checking routine information, retrieving specific content, completing fixed navigation paths, and filling structured form-like workflows. Prior studies characterize smartphone interaction as highly habitual, with repetitive checking behavior and app-launch patterns that are strongly shaped by recurring contexts such as time and location [? ?]. Public mobile GUI datasets and benchmarks also reflect a similar pattern. For example, the largest-scale mobile GUI dataset to date, AITW, contains 715K episodes but only around 30K unique instructions [?]. Therefore, although per-step reasoning for open-ended tasks is crucial for high task completion rates, many realistic mobile-agent workloads are better viewed as repeated executions of recurring workflow patterns.

2.2 Reducing Runtime Costs

Prior studies on reducing the runtime costs of GUI agents primarily fall into several categories, which are summarized in Table 1.

Robotic process automation (RPA)-based automation systems treat mobile tasks as fixed workflows by recording and replaying user demonstrations [? ? ? ?]. These systems construct replayable execution traces through human-driven app exploration before task execution. However, the resulting execution traces are tightly bound to the recorded workflow and GUI state. Workflow deviations, such as unexpected pop-ups or alternative entry states, can break the recorded path, while layout changes can invalidate recorded coordinates or UI selectors.

Memory-based GUI agents reduce runtime costs by reusing memory constructed from prior app exploration or first-time task execution. For example, M3A [?] stores prior actions and their reasoning in prompts to guide later execution. AutoDroid [?] and AppAgent [?] construct app-specific memory, such as UI transition graphs, through automated exploration and retrieve this memory during execution. MobileGPT [?] extends this idea with hierarchical app memory that stores reusable procedures and subtasks summarized from prior executions. MobiMem [?] reduces repeated GUI-agent reasoning by reusing prior executions through experience memory, which abstracts task traces into reusable templates, and action memory, which caches low-level action sequences for direct replay. **Program-guided agents** [? ? ? ?] further reduce runtime costs by translating task knowledge or prior demonstrations into executable scripts. These scripts allow repeated GUI operations to be completed through deterministic program execution with fewer model invocations. However, such memory and executable programs are typically built around previously observed workflows. When live execution deviates from stored experience, existing systems often require online model reasoning to reinterpret the current state, recover the workflow, or adapt the memorized operation. In some cases, such deviation can make the entire stored memory unusable for the current execution. These runtime mismatches substantially reduce the benefit of memory reuse in dynamic mobile environments.

Another line of studies, **per-step inference acceleration methods**, retain the traditional observe–reason–act loop while reducing the cost of each individual decision. These methods accelerate either action generation or input processing. V-Droid [?] replaces autoregressive action generation with verifier-based ranking over a discretized action space, substantially reducing decision latency. ShowUI [?] accelerates VLM inference through GUI-guided visual token selection. Other studies reduce inference costs by pruning redundant visual or historical context, including masking-based GUI element pruning, history compression [?], and position-preserving GUI token selection [?]. However, these methods still invoke the model at every interaction step, causing inference costs to accumulate throughout long-running GUI tasks.

Overall, existing replay and memory reuse approaches remain fragile to dynamic mobile environments because their stored execution knowledge is collected from previously observed workflows and GUI states. This calls for adaptive replay that can preserve memory execution despite runtime deviations.

2.3 Goals and Challenges

Our goal is to enable mobile GUI agents to reuse past executions in dynamic environments whenever possible. Achieving this goal requires the system can adapt to both workflow- and GUI-state-level dynamics: (1) Workflow-level adaptation allows the agent to reuse applicable parts of prior executions when the live workflow changes, and to quickly resume memory-guided execution once a known GUI state is reached. (2) GUI-state-level adaptation allows a memorized action to be re-grounded to the correct target when the live GUI state changes. Realizing these capabilities raises two fundamental challenges.

Challenge #1: Fine-grained memory representation and fast retrieval. The first challenge is to support flexible memory reuse with low retrieval overhead. Existing GUI-agent memory often stores complete trajectories or fixed subtasks, causing reusable execution segments to be discarded when only part of the workflow changes. This calls for fine-grained capability Functions that can be independently reused and composed into higher-level tasks. At runtime, the system must also quickly locate applicable Functions from

the current GUI state. However, visual-embedding-based state matching can introduce substantial computation and latency on mobile devices. Efficient replay therefore requires both fine-grained memory representation and lightweight per-state retrieval.

Challenge #2: Adaptive action re-grounding. The second challenge is to reliably transfer a memorized action to the live GUI with low runtime overhead. Even for a matched GUI state, the target component may change in position, appearance, or attributes across devices and app versions. Resource IDs, for example, change in approximately 50% of corresponding widgets across app releases [**?**], while lexical identifier matching achieves only 28% Top-1 recovery on real-world view-identification failures [**?**]. Re-invoking an LLM/VLM can resolve such mismatches, but incurs nontrivial overhead: even a small 0.8B model takes over 10s for on-device execution and around 3s via a cloud API for a single re-grounding decision[1]. Practical replay therefore requires lightweight action re-grounding using semantic and structural GUI evidence.

To address these challenges, we design OmniFlow, an adaptive replay system that reuses prior successful executions whenever they remain applicable to the current GUI. It supports fine-grained memory reuse and efficient action adaptation, while invoking VLM reasoning only for portions that cannot be reliably replayed. The next section presents the design of OmniFlow.

3 OmniFlow Design

3.1 System Overview and Workflow

OmniFlow is an adaptive replay system that reuses historical GUI executions whenever they can be reliably applied. To do so, OmniFlow consists of two core components: (1) the Omni Representation Layer, which organizes historical executions into fine-grained, composable functions and supports fast per-GUI-state retrieval; and (2) the Flow Execution Layer, which executes recalled functions and re-grounds their recorded actions to the live GUI state. Figure 2 shows the workflow of OmniFlow. Overall, it has two stages: offline memory construction and online memory reuse.

Offline memory construction. OmniFlow first analyzes successful historical executions offline using a VLM. It decomposes each execution into fine-grained, reusable functions and organizes their GUI states and transitions into a transition graph. Each function captures an independently executable operation, while multiple Functions can be composed to accomplish higher-level tasks. Meanwhile, OmniFlow constructs a lightweight representation for each GUI state from its view hierarchy to support fast online retrieval. The resulting graph and functions form the reusable execution memory.

[1]Model: Qwen3.5-0.8B-W4A16; Execution environment: Llama.cpp on OnePlus 12 using CPU backend.

Online memory reuse. During task execution, OmniFlow continuously matches the live GUI state against the stored memory and retrieves applicable functions. When a function is available, the Flow Execution Layer directly executes its recorded actions without invoking the VLM for every step. Since the live GUI may differ from the recorded one, each action target is re-grounded using lightweight multimodal element matching together with local structural and spatial context. Only reliably matched actions are executed.

When no applicable function is found, or an action cannot be reliably re-grounded, OmniFlow returns control to the VLM for traditional step-wise reasoning. After the VLM handles the unmatched interaction, OmniFlow immediately searches the memory again. Once a memorized GUI state is reached, OmniFlow resumes function-based replay. This alternating execution allows OmniFlow to preserve memory reuse across dynamic workflows while invoking expensive VLM reasoning only when necessary.

Agent integration. Architecturally, OmniFlow operates as a plug-and-play cache layer for existing GUI agents using an `observe` interface to capture the current UI hierarchy and screenshot, and an `act` interface to dispatch GUI actions. The `observe` hook triggers function retrieval before the agent's planning phase, exposing recalled functions alongside primitive agent actions (e.g., click, input, scroll). When a function is selected, the `act` hook uses the Flow Execution Layer to project the recorded action to the current GUI. This interface-based boundary allows OmniFlow to be integrated into existing GUI agents without modifying their original prompts, planners, or action decoders.

3.2 Omni Representation and Retrieval

The Omni Representation Layer converts successful historical executions into fine-grained executable memory that can be efficiently reuse from the current GUI state. Historical executions are naturally collected when a user or a GUI agent successfully completes a task. For each execution, OmniFlow records the sequence of actions together with the GUI state before and after each action. It then processes these executions offline in two steps: extracting reusable functions and constructing lightweight GUI representations. The resulting functions and GUI states are organized into a transition graph for online retrieval.

Constructing reusable functions. As shown in Figure 3, OmniFlow converts successful executions into reusable Functions through three stages: segment discovery, parameter abstraction and binding, and compilation.

OmniFlow first constructs GUI-state-level functions offline through a VLM harness over successful historical executions. Our key insight is that executable memory should be organized around individual GUI states, so that reusable operations can be entered and resumed from intermediate points of historical executions. Therefore, the harness takes the recorded task instruction, each GUI state, the executed

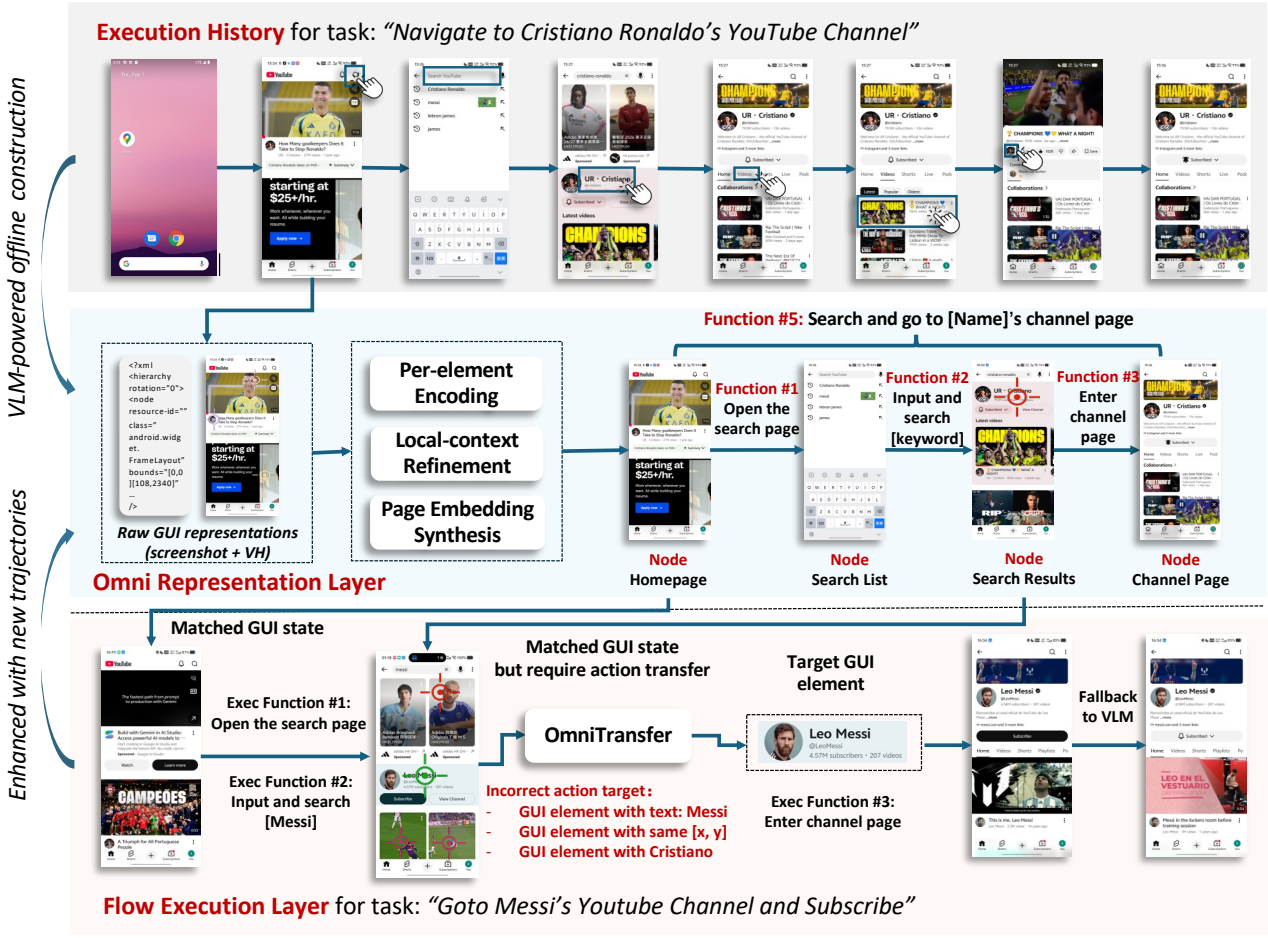


Figure 2. End-to-end demonstration of OmniFlow.

action, and the subsequent GUI state as input. Based on this context, the VLM summarizes each transition into a function that preserves its entry state, recorded action, and resulting state for subsequent retrieval and action re-grounding. To support task-level retrieval, OmniFlow further composes consecutive GUI-state-level functions into higher-level functions according to the recorded task intent. Each composed function summarizes a reusable execution path while retaining its constituent functions for fine-grained replay.

Lightweight GUI-state representation. Fast function retrieval requires OmniFlow to determine which memorized GUI state corresponds to the current screen. Applying a visual encoder to every observed screen introduces substantial latency and computation overhead on mobile devices. Our key observation is that state retrieval only needs to determine whether two GUI states expose similar functional structures, rather than fully understanding their visual content. The GUI view hierarchy already provides lightweight cues for this purpose, including widget properties, page structure, and element geometry. We therefore encode each GUI element using these hierarchy cues together with a lightweight visual signal. The resulting representations are aggregated into learned structural subspaces and concatenated into a compact page-level embedding. §3.3 further details the architecture. The lightweight page embedding also serves as the key for organizing historical executions. GUI states with matching embeddings are associated with the same graph node, which stores the reusable functions available from that state. Function executions connect their entry and resulting states, forming a fine-grained transition graph across historical executions. This organization allows multiple executions to contribute reusable functions to shared GUI states.

Fast function retrieval. At runtime, OmniFlow retrieves reusable functions using both the task instruction and the current GUI state. It first uses the task instruction to identify relevant composed functions, which narrows the search from the entire transition graph to historical execution paths with similar task intent. Within these candidate paths, OmniFlow computes the lightweight page vector of the current GUI and matches it against the stored graph nodes. The matched node directly exposes the GUI-state-level functions available from the current state, which are then passed to the Flow

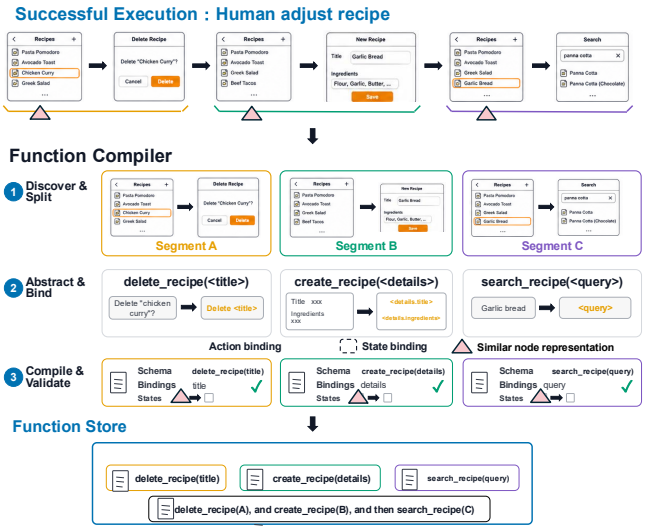


Figure 3. Offline function augmentation Our harness performs offline augmentation in three stages: (1) *Discover and Split*, which extracts reusable segments from successful execution trajectories and separates recurring recovery operations as shared Checkers; (2) *Abstract and Bind*, which converts concrete segments into parameterized Functions and binds their parameters to GUI actions and UI states; and (3) *Compile and Validate*, which validates and registers the resulting Functions in the Function Store for reuse across tasks and devices.

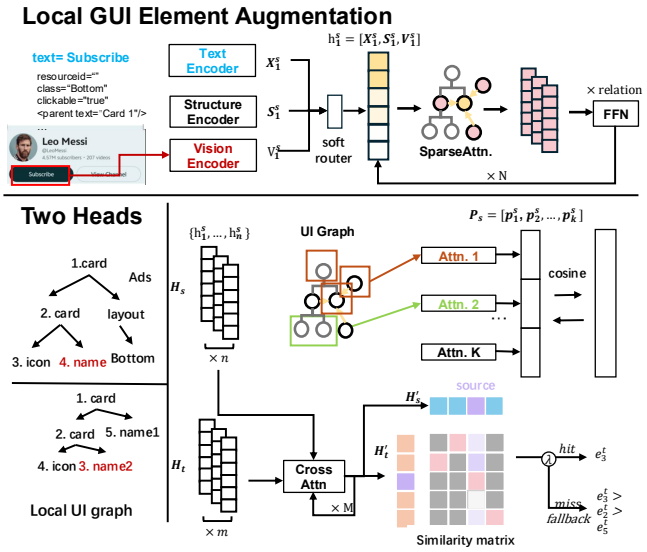


Figure 4. Architecture of OmniTransfer. The first stage encodes the GUI screenshot and view hierarchy into dense element representations and refines them through sparse local aggregation. The second stage uses two task heads to synthesize a page embedding and compute element-level transfer similarities for action re-grounding.

Execution Layer for replay. This two-stage retrieval avoids searching a large memory graph exhaustively while enabling fast re-entry into historical execution from intermediate GUI states. Specifically, OmniFlow ranks candidate functions by combining the semantic relevance of their name and description to the task instruction with their entry-state similarity to the current GUI. When the top-ranked function exceeds a confidence threshold, it is passed to a lightweight router that verifies its executability and instantiates the required parameters. This design avoids repeated long-context reasoning for recurring GUI situations and reduces token consumption.

3.3 Flow Replay and Action Re-Grounding

The Flow Execution Layer replays retrieved functions on the live GUI state. Since the recorded action target may change due to state-level dynamics, OmniFlow does not directly reuse its original coordinate or selector. Instead, it employs OmniTransfer, a lightweight learned model that locates the corresponding target element on the live GUI state before execution. The model takes the recorded target GUI element, its source GUI state, and the live GUI state as input. It first constructs an encoded GUI graph through multimodal element encoding and local-context refinement, then derives a page embedding and an element-matching result that identifies the corresponding target in the live GUI state. Figure 4 demonstrates the architecture of OmniTransfer.

Multimodal GUI element encoding. OmniTransfer first encodes each GUI element in the recorded and live GUI using its text, visual crop, hierarchy-derived widget attributes, and geometric features. We optimize for discriminability rather than semantic learning so Since these modalities provide different amounts of evidence across elements, a lightweight modality router assigns element-specific weights to the available modalities and masks missing ones during fusion. For example, text can dominate for a labeled button, while visual information can be more useful for an unlabeled icon. The same encoder is applied to both recorded and live elements, producing comparable element representations for subsequent matching.

Local-context calibration. Element-level features may be insufficient when multiple widgets have similar text, appearance, or widget types. Our key insight is that an element's role within its local structural and spatial context provides more stable evidence for cross-GUI correspondence than its individual attributes alone. For example, modern GUIs contain many placeholder cards and layout-oriented views that are visually meaningful but weakly represented in the view hierarchy, which often provides little more than a generic ID and bounding box. OmniTransfer therefore augments each element representation with its local structural and spatial context. For each GUI element, it gathers

informative neighbors from the GUI hierarchy and layout, including its parent, siblings, and nearby elements. Their representations are aggregated with learned relevance weights to emphasize neighbors that help distinguish the target. For example, two visually identical action icons can be differentiated by the labels or containers surrounding them. Restricting calibration to local neighborhoods avoids expensive global interaction over all elements on the page while preserving the context needed for cross-GUI correspondence.

Page embedding head. The context-calibrated node representations are processed by K parallel attention modules. Each module learns a distinct weighting over GUI elements, producing a complementary summary of the page. The weighted node representations are aggregated into K page-level slots and concatenated to form the final page embedding. The same shared head is applied to both source and target pages, and their embeddings are compared using cosine similarity. In our implementation, $K = 8$ and each slot has 128 dimensions, producing a 1,024-dimensional page embedding.

Target matching head and verification. After context calibration, OmniTransfer locates the recorded action target on the live GUI by comparing its representation with those of candidate live elements. The matcher computes correspondence scores and ranks the candidates for each recorded target. To reduce ambiguous matches, OmniTransfer normalizes correspondence in both source-to-target and target-to-source directions, favoring element pairs that are mutually consistent across the two GUIs. The highest-ranked candidate is accepted only when its confidence satisfies the execution criterion. Once accepted, the recorded operation is applied to the matched live element instead of its original coordinate or selector.

When OmniTransfer cannot identify a reliable target, OmniFlow stops replay and returns control to the VLM for step-wise action generation. After the VLM handles the unmatched interaction, the Omni Representation Layer matches the resulting GUI against the stored transition graph again. If a reusable state is found, OmniFlow retrieves the corresponding function and resumes replay. This allows VLM reasoning to handle only interactions that cannot be reliably transferred, while memory-based execution resumes whenever a reusable GUI state becomes available.

Closed-loop flow execution. A retrieved function is executed step by step rather than replayed atomically. After the planner selects a function, OmniFlow observes the current GUI before each action and uses OmniTransfer to re-ground its target on the live state. If the correspondence satisfies the execution criterion, the action is executed, and the resulting GUI is observed before continuing. If re-grounding fails, OmniFlow returns control to the VLM for step-wise reasoning, then resumes memory-based execution when a reusable function becomes applicable.

4 Implementation

OmniFlow operates as a plug-in cache layer for existing mobile GUI agents, enabling seamless integration while augmenting their execution with reusable functions. We use Python 3.11 for OmniFlow's runtime implementation, which contains approximately 16,752 lines of code. We use Model Context Protocol (MCP) to integrate OmniFlow runtime with existing mobile GUI agents. During execution, the recalled functions are passed to the agent as MCP tools. When a function is executed, the runtime invokes the OmniTransfer inference pipeline to map the recorded action to the current GUI state. If the action transfer process fails, control is passed back to the host agent using the exception handling signals.

The OmniTransfer model is trained using PyTorch. The final compact checkpoint contains 1,194,582 parameters (1.2M) and requires roughly 4.8 MB for its compiled float32 parameter storage. It is deployed for on-device inference through a lightweight NumPy runtime rather than a full deep-learning framework. During the online `act` phase, the matcher executes on the mobile CPU and averages 118.5 ms per transfer.

- *Training Data.* To elicit the model's capability for action transfer under dynamic GUI environments, we train it on the ASE dataset [**?**], which provides supervised cross-platform action-transfer data. The training split utilizes 835 source–target GUI pairs, yielding 5,944 samples in total. The validation split contains 115 source–target GUI pairs, corresponding to 1,566 directed widget-mapping samples.
- *Training Setup.* The model was trained from random initialization (seed 17) for 15 epochs using the AdamW optimizer. Hyperparameters were set to a learning rate of 3×10^{-4} and a weight decay of 0.05. Batch sizes were configured to 4 for ASE matching updates and 8 for page batches. Full model training took 0.82 GPU hours on a single NVIDIA GeForce RTX 4090.

5 Evaluation

We evaluate OmniFlow from three perspectives. First, we conduct an end-to-end evaluation of mobile GUI agents to examine whether adaptive memory reuse remains effective under dynamic execution environments. Second, we use micro-benchmarks to evaluate the effectiveness of OmniTransfer in action re-grounding. Finally, we evaluate OmniFlow on physical mobile devices to characterize the runtime overhead of memory-augmented execution under realistic deployment settings.

5.1 Experimental Setup

Constructing Dynamic Execution Environments. Our goal is to evaluate OmniFlow under diverse mobile dynamics while retaining controlled execution and deterministic evaluation. We therefore build our end-to-end evaluation on AndroidWorld [**?**], a benchmark containing 116 tasks across 20 Android applications. Starting from its standardized task

definition and environment setup, we construct dynamic execution environments by varying both task instances and device configurations. Specifically, each task is first executed on the pre-defined emulator configuration, running Android 13 at 720 × 1280, with a predefined seed to collect the source memory. We then evaluate the collected memory with another seed under three target configurations:

- A standard phone (*Standard*): Pixel 6 Pro running Android 13 at 1440 × 3120 screen resolution
- A foldable phone (*Fold*): 7.6-inch foldable device running Android 14 at 1768 × 2208 screen resolution
- A tablet (*Tablet*): 10.1-inch WXGA tablet running Android 13 at 1280 × 800 screen resolution

The changed seed produces different task parameters, while the device configurations introduce variations in screen size, aspect ratio, layout, resolution, and Android version. This setup exposes OmniFlow to controlled execution dynamics while preserving deterministic evaluation for each configuration. AndroidWorld controls task initialization and success validation, while device launch, GUI observation, and GUI actions are executed through the OmniFlow interaction interfaces.

Baselines. We compare the following representative memory-based mobile GUI agents.

- *MobileGPT* [?] organizes successful executions into a hierarchical task–subtask–action memory, with parameterized subtasks that can be recalled and adapted across executions.
- *AutoDroid* [?] randomly explores an app to construct a GUI transition graph, then synthesizes simulated-task memory linking task intents with relevant GUI states, elements, and actions for runtime retrieval.
- *AppAgent* [?], similar to AutoDroid, builds an app-specific reference document through autonomous exploration or human demonstrations, recording functions and action effects of each GUI element to guide step-wise execution.
- *OmniFlow* uses M3A [?] as the VLM-based backend for step-wise reasoning and action generation, with OmniFlow added as the memory-reuse layer for recalling and executing reusable functions.

For methods that directly accept prior executions, we use the same successful execution trace to construct their memory for a fair comparison. For AutoDroid, we follow its original experience-construction and explore a complete UTG for every evaluated AndroidWorld application on the source device. For MobileGPT, we additionally perform its native cold-start procedure on the source device, using successful source executions as teacher trajectories. All baseline methods use identical task initialization and success criteria, and retain their original execution loops and prompts. Qwen3.6-Plus [?] (`qwen/qwen3.6-plus` via the OpenRouter platform) is used as the common backend VLM.

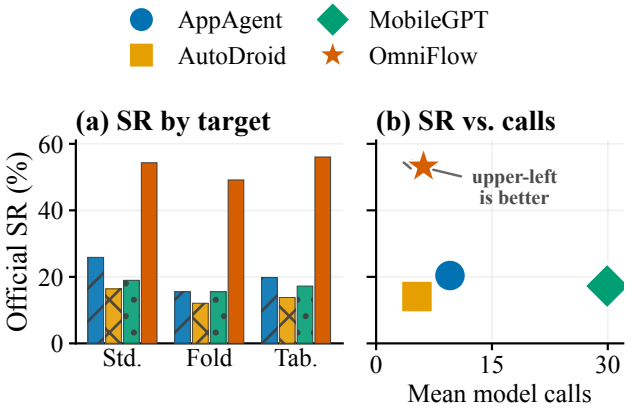


Figure 5. (a) Success rate across Standard, Fold, and Tablet environments. (b) Overall success rate versus average model invocation times over successful episodes.

Micro-benchmarks. We further evaluate the two key capabilities of OmniFlow independently. The ASE benchmark [?] measures the action re-grounding capability of OmniTransfer across cross-platform GUI variations, with rule-based element matching as baselines. We report top-1 Transfer accuracy. The PAPT benchmark [?] evaluates the GUI-state representation used for function recall across phone–tablet layout changes, with screenshot- and view-hierarchy-based representations as baselines. We report top-1 page retrieval accuracy.

Physical-device experiment. We additionally deploy the complete online runtime on a OnePlus Ace 3 with approximately 16 GB of memory and measure its end-to-end execution cost. We compare step-by-step VLM execution with OmniFlow-enhanced execution under the same tasks, model backend, GUI observation, and device interaction interface. We report task completion time, model calls, prompt and completion tokens, and peak memory, and further break down execution time into GUI encoding, function recall, action transfer, physical action, and VLM processing. We also measure device energy consumption using the Android battery charge counter and voltage when both are available.

5.2 Memory Reuse Efficiency in End-to-end Execution Environments

In this section, we evaluate OmniFlow's end-to-end task-completion capability under dynamic execution environments. Using AndroidWorld, we instantiate the same task suite across multiple device configurations and task parameters.

Overall effectiveness. As shown in Figure 5, OmniFlow achieves an overall success rate (SR) of 53.2% (185/348) across the Standard, Fold, and Tablet environments, compared with

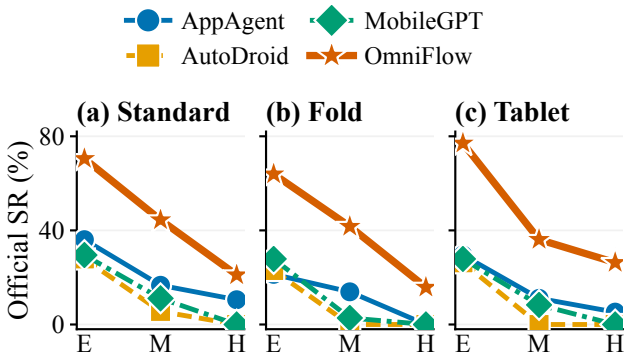


Figure 6. Success rate categorized by AndroidWorld task difficulty levels and execution environments. E: Easy; M: Medium; H: Hard.

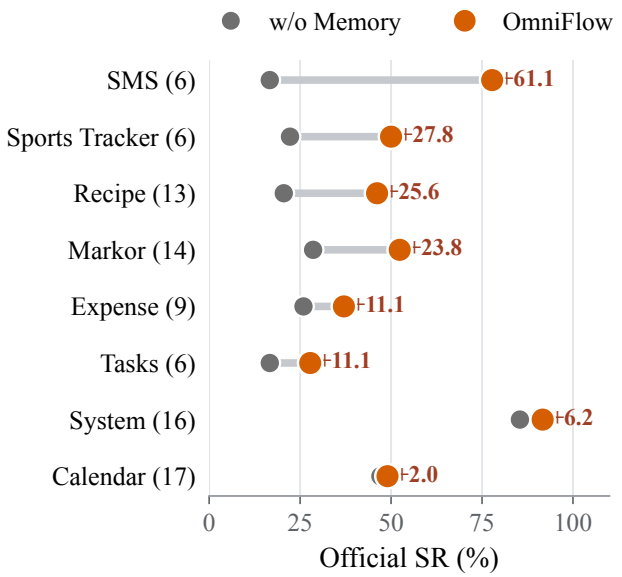


Figure 7. Effect of Function Memory across application families. We report every application family containing at least six official tasks.

37.1% (129/348) for the same online executor without memory reuse. Its SR remains consistent across the three environments, reaching 54.3%, 49.1%, and 56.0%, respectively.

Robustness across task difficulty. Figure 6 shows that the improvement persists across all difficulty levels. OmniFlow achieves 70.5%, 40.7%, and 21.1% SR on easy, medium, and hard tasks, compared with 58.5%, 19.4%, and 1.8% for the online executor. The lower success rate on hard tasks reflects the prevalence of content-dependent workflows, such as browser and drawing tasks, where fewer deterministic segments can be transferred and earlier fallback to online reasoning is required.

Fixed memory granularity limits reuse under workflow variations. Existing memory representations struggle when only part of a recorded workflow remains reusable. AutoDroid achieves an overall SR of 14.1% (49/348), which drops to 6.3% (18/288) on tasks requiring parameter substitution, 2.8% on action repetition, and 5.6% on complex GUI decisions. MobileGPT exhibits a similar trend: its overall SR of 17.2% (60/348) falls to 7.6% on parameterized tasks and to 0% on repetition, complex-GUI, and longer-than-ten-step tasks. These results show that memory organized at a fixed granularity is difficult to partially reuse when workflow structure changes.

OmniFlow is more robust in memory retrieval. Existing page representations struggle to remain reliable under layout changes, with loosely bound representations tending to over-match and device-specific representations tending to under-match. AutoDroid's transition-graph memory is loosely bound and over-matches: across 501 diagnostic executions, it reports memory reuse in 500 cases, yet only 47 of these executions succeed, because a retrieved state does not ensure that the associated action remains executable under a different layout, device, or interaction history. MobileGPT addresses this uncertainty by re-verifying the current state with an embedding model at every step, resulting in 29.9 model calls per successful episode. At the other extreme, the third baseline uses a device-specific structural key and under-matches: across 4,095 decision rounds, only 928 (22.7%) retrieve any memory entry.

Online inference cost and function reuse. On the 83 task–device pairs solved by both OmniFlow variants, function memory reduces the average number of model calls from 6.70 to 3.99 (40.5%) and token usage from 26.4k to 13.0k (50.8%). Across all tasks successfully completed by OmniFlow, including episodes requiring online recovery, it still reduces model calls by 24.1% and token usage by 35.9%. Among 1,644 reusable GUI actions in 185 successful episodes, 946 are served directly from memory, corresponding to a 57.5% action-reuse rate. These results show that OmniFlow replaces a substantial fraction of repeated online reasoning with verified action reuse. We report latency separately under a controlled paired evaluation.

Storage overhead. OmniFlow compiles 116 recorded executions into 190 reusable functions, whose definitions occupy only 0.53 MiB in total, or 2.88 KiB per function. By retaining only execution-relevant evidence, the stored memory reduces 642.68 MiB of raw execution data to 279.84 MiB, a 56.46% reduction. Offline function construction takes 17.58 s and generates 338 output tokens per function on average. For cross-device sharing, OmniTransfer represents each GUI page with an average FP16 payload of 53.9 KiB, incurring 13.27 ms of paired communication overhead. These results show that OmniFlow can construct and distribute reusable experience with modest storage and communication costs.

Failure analysis.

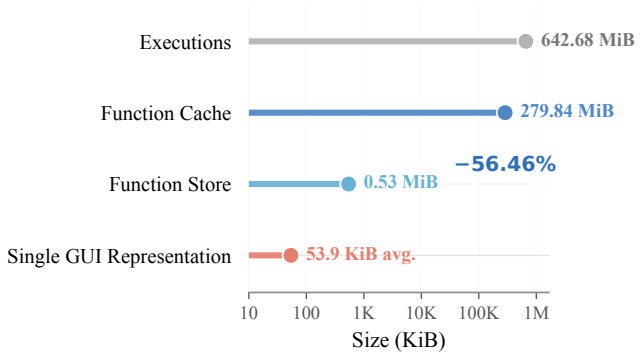


Figure 8. Storage overhead of OmniFlow. Function Cache translates all execution histories to compact GUI representation, whereas the Function Store contains all reusable Functions. The Single GUI Representation reports the mean encoded payload per GUI page.

In `ContactsNewContactDraft`, an unexpected onboarding pop-up appeared on the target devices but was absent from the source execution. A shared Checker detected and dismissed the pop-up, allowing the original Function to continue without modification. By contrast, AppAgent failed the official validator on all three devices, while MobileGPT required 21–40 model calls to complete the task.

In `RecipeAddSingleRecipe`, a Function constructed from adding "Turkey and Cheese Panini" was reused to add "Avocado Toast with Egg." The Agent-authored bindings replaced the recipe-specific fields, while OmniTransfer relocated the recorded actions to the new layouts. OmniFlow successfully replayed the complete procedure on all three devices with only two model calls. In contrast, both MobileGPT and AppAgent failed the official validator on all three devices; MobileGPT executed no action on Fold and Tablet, while AppAgent exhausted its action budget on Standard and Fold.

Among failed episodes, 62.0% ran the selected Function to completion without any action-level failure, yet still failed the official task validator. Successful execution of the recorded action sequence is therefore insufficient to guarantee semantic task completion. A Function encodes only the reusable portion of a workflow. Task-specific parameter grounding, observation-dependent decisions, fallback handling, and completion verification remain the responsibility of the online agent. Residual failures therefore cannot be attributed to Action Transfer alone; they also reflect incomplete Function coverage and limitations in the online agent's perception and reasoning.

5.3 Micro-benchmark for OmniTransfer

In this subsection, we evaluate the two core capabilities of OmniTransfer independently: (1) **page recall**, which measures whether OmniTransfer can recognize functionally equivalent GUI states across devices and retrieve the corresponding historical states; and (2) **action re-grounding**, which measures whether it can relocate recorded GUI actions to the corresponding elements in the current device state. The main results are presented in Figure 9.

Table 2. Unified component ablation on the ASE test set.

Configuration	Top-1	Top-3	Top-5	Δ Top-1
Full configuration	**82.74**	**93.70**	**95.65**	0.00
w/o visual crop encoder	74.81	87.87	92.15	−7.93
w/o local-context calibration	59.41	73.17	79.08	−23.33
w/o matrix score refinement	73.41	86.08	89.74	−9.33

Page recall. The learned page head reaches 93.75% R@1 and 99.22% R@5, matching the Qwen3-VL screenshot embedding while avoiding remote inference. Its mean cold encoding time is 32.97 ms per page, compared with 1004.21 ms for the screenshot baseline and 214.07 ms for the XML embedding baseline on API. Therefore, the page head preserves the strongest retrieval accuracy while reducing mean latency by 30.5× and 6.5×, respectively. It also uses no API requests or tokens, which makes the embedding suitable for continuous runtime state lookup.

Action re-grounding. We evaluate action grounding on ASE test set, which comprises 98 page pairs, 196 mapping directions, and 1,286 reviewed correspondences. Five baselines are designed to isolate the evidence available without task-specific matcher training. Text matching achieves only 36.86% Top-1 accuracy, as labels may be missing, duplicated, translated, or attached to different XML nodes across platforms. Incorporating layout scaling raises Top-1 to 44.95%, yet remains brittle when Android and iOS have different view hierarchies. A handcrafted text-layout fusion improves Top-1 to 62.75% and R@5 to 85.07%, but it relies on fixed heuristics and cannot dynamically determine which modality is trustworthy for each individual widget. A stronger visual baseline, LightGlue [**?**], matches SuperPoint keypoints between complete source and target screenshots and pools the resulting correspondences into target XML bounding boxes for node ranking. Although it improves coverage to 69.83%, it still attains only 29.32% Top-1, 39.35% R@3, and 41.37% R@5. Moreover, LightGlue relies purely on visual appearance–ignoring node text, semantic roles, hierarchy, and sibling relationships—while requiring a heavy inference cost of 1,185.14 ms per page pair on the Apple M4 CPU.

In contrast, our OmniTransfer ranks all valid target nodes by jointly leveraging textual, visual, XML-structural, and local relational evidence. By learning to fuse these heterogeneous signals, it overcomes the limitations of both heuristic fusion and unimodal visual matching. OmniTransfer achieves 82.74% Top-1, 93.70% R@3, and 95.65% R@5, while the action head adds only 22.30 ms on average on Apple M4 CPU, enabling efficient and privacy-preserving deployment.

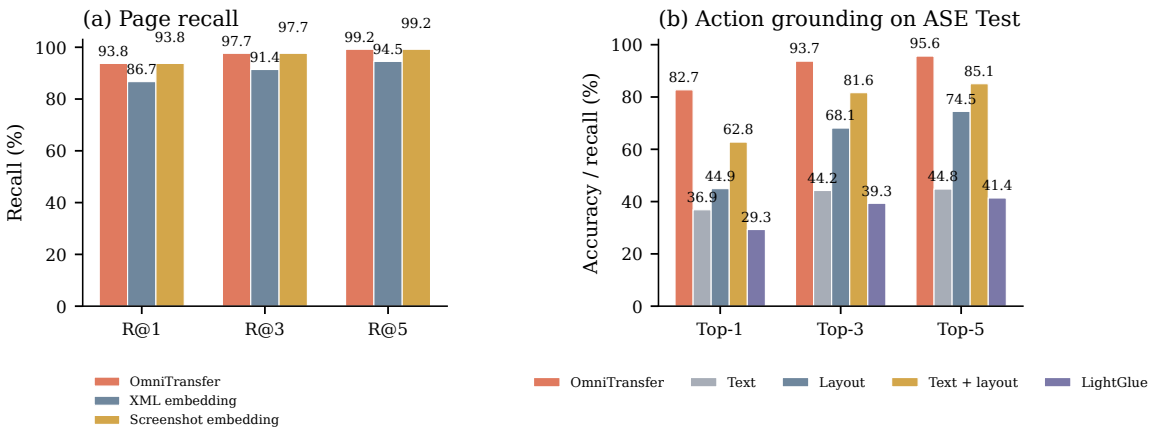


Figure 9. Page recall and action grounding with OmniTransfer. (a) Page-recall accuracy on PAPT. (b) Action re-grounding performance on ASE.

Ablations. Table 2 presents the component ablation results. The largest decrease occurs when the model is reduced to multimodal GUI element encoding only: Top-1 falls by 23.33 percentage points. This shows that individual element descriptors are insufficient when cross-platform pages contain duplicated or missing labels. Removing learned relevance-weighted aggregation lowers Top-1 by 7.62 points, confirming that local neighbors are most useful when their contributions are selected adaptively rather than merged uniformly. Removing local score refinement in target matching and verification causes a 9.33-point drop, indicating that candidate comparison and bidirectional verification remain necessary after representation learning. Finally, masking the visual crop reduces Top-1 by 7.93 points, demonstrating that visual evidence provides complementary information for unlabeled or visually distinctive controls. Together, these results support the design of the action head as a combination of multimodal GUI element encoding, local-context calibration, and target matching and verification.

5.4 Execution Overhead

We additionally measure the on-device cost of executing a recalled Function on a physical device. The measurement covers the four operations required to replay one recorded action: acquiring the current GUI state, encoding its GUI representation, re-grounding the recorded action, and executing the action with GUI stabilization.

Observation/action decomposition. AndroidWorld and many ADB-based agents acquire GUI states and execute actions through repeated host–device communication, making this control path a major source of per-step latency. OmniFlow instead performs state acquisition and action dispatch through a resident on-device service while preserving the same observation format and action semantics. In our controlled experiment, observation latency decreases from $1,334 \pm 32$ms to 302 ± 72ms, and action latency decreases from $1,229 \pm 56$ms to 556 ± 179ms. In particular, state acquisition drops from 1,231ms to 207ms, whereas host-side processing remains comparable (76ms versus 73ms). Thus, the speedup primarily comes from reducing repeated host–device communication, lowering the assembled observation-plus-action latency from 2,563ms to 859ms.

On-device GUI representation construction. We first evaluate whether a general-purpose vision-language embedding model can support continuous GUI encoding. In our deployment-feasibility test, the W4-quantized Qwen3-VL-Embedding model requires 18.4s for a short HTML-like input and approximately 30.8s for a screenshot, while occupying 2.7–2.8GB of resident memory. In contrast, OmniTransfer's lightweight page encoder processes 100 distinct real GUI pages from 30 applications in 65.9 ± 27.1ms per page, with an active energy cost of 0.046mWh. This shows that the lightweight page head is suitable for continuous on-device state lookup, whereas the general-purpose embedding model is not.

Action re-grounding on device. We separately measure the cost of mapping a recorded action from its historical source state to the current GUI. Across 50 distinct same-application state pairs from 10 applications, OmniTransfer completes action re-grounding in 110.3ms on average, with an active energy cost of 0.061mWh per mapping. Together, page encoding and action re-grounding require only 176.2ms, accounting for 11.1% of the assembled 1.58s execution loop. Thus, learned re-grounding supports online execution and is not the dominant runtime bottleneck.

6 Discussion

Large-scale deployment. OmniFlow supports large-scale experience sharing. A shared function store can be hosted remotely, while the device locally encodes the current GUI and uploads only its GUI representation for function recall. Users may alternatively maintain a private local function

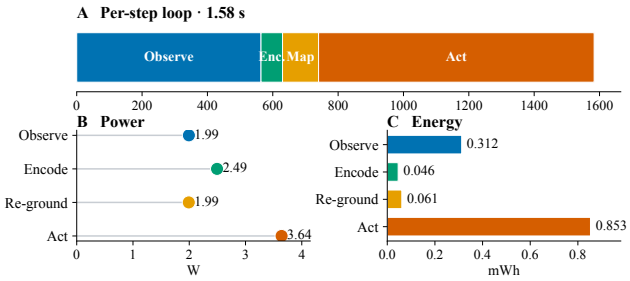


Figure 10. On-device Function-replay overhead on OnePlus. (A) Component latency, (B) incremental power above idle, and (C) active energy per call. Latencies are independently measured component means.

store. This deployment choice does not change the function representation or the online execution logic; it only changes where reusable functions are stored and distributed. OmniFlow decouples Function storage from on-device execution. So a remote Function Store can distribute reusable experience across devices, while each device locally encodes its current GUI and sends only the representation required for Function Recall. For privacy-sensitive deployments, the same Function Store can remain entirely on device.

Semantic binding and generalization. OmniFlow enables model-free execution by binding semantic operations to node-level GUI actions during function construction. A reusable function therefore specifies how its task semantics, parameters, and intermediate decisions map to concrete GUI operations. Given such a binding, OmniTransfer can efficiently relocate the recorded operations across devices. This representation naturally composes with scripts and programmatic methods as a reusable execution unit. For operations whose semantic-to-action mapping has not been constructed, OmniFlow falls back to the VLM to interpret the live content and determine the corresponding GUI action. Parameterization supports variations in known values, while content-dependent decisions may still require runtime reasoning. This design shifts semantic grounding from repeated online inference to function construction, enabling efficient deterministic replay for recurring operations. Future work could generalize semantic-to-action bindings across previously unseen decisions and execution contexts.

7 Related Work

Step-wise and verifier-based GUI agents. VLM-based mobile GUI agents usually follow an observe–reason–act loop: each screen update is converted into a new model decision, and the system issues one concrete GUI action before observing again. V-Droid [**?**] improves this loop by shifting the LLM role from action generation to action verification over a discrete action set, reporting 4.3 s per step and 59.5% success on AndroidWorld. VeriSafe [**?**] similarly focuses on action safety by checking LLM-generated actions against logic constraints derived from the user's goal. These systems improve the cost or correctness of individual steps, but repeated routines still pass through online decision or verification for each interaction. OmniFlow is complementary: a step-wise or verifier-based agent remains the fallback, while matched repeated Functions are served as executable Function cache hits.

Memory-based and programmatic GUI agents. AutoDroid [**?**], MobileGPT [**?**], and AppAgent [**?**] reduce repeated GUI-agent work by building app- or task-specific memory from exploration and prior executions. AndroidWorld's M3A baseline [**?**] also exposes the benefit of carrying previous actions and reasoning into future attempts. These systems reduce per-goal LLM cost when a warm-start match is found, but their memory is primarily used to guide online reasoning or replay app-specific subtasks. Reuse is limited by per-app and per-device cold-start exploration and UI-attribute-based action grounding: XML selectors break on WebViews, custom-rendered feeds, and cross-device layout variation, which forces repair, new exploration, or online LLM calls. ActionEngine [**?**] and AutoDroid-V2 [**?**], as well as AgentProg [**?**], move toward programmatic execution, building app-level state knowledge or generated scripts to reduce online planning. Their reuse unit is a broad app model or complete task program that requires substantial upfront exploration and assumes a stable live app structure.

GUI record-and-replay. Recent GUI record-and-replay systems extend replay beyond a single stable device or platform. Cross-device Android replay adapts recorded executions across device configurations [**?**], while cross-platform migration and replay systems transfer tests across Android, iOS, and visual interfaces [**? ?**]. These works show that GUI reuse must handle layout drift, widget mismatch, and effect preservation. OmniFlow uses replay as part of the successful experience reuse problem.

8 Conclusion

We present OmniFlow, an adaptive replay system for reusing successful GUI executions across dynamic tasks and devices. OmniFlow converts historical trajectories into fine-grained, composable functions that can be retrieved from intermediate GUI states. OmniTransfer further adapts recorded actions to changing layouts through lightweight state retrieval and action re-grounding. When reuse is unreliable, OmniFlow falls back to VLM reasoning and resumes replay once a reusable state is reached. OmniFlow achieves a 53.2% task success rate, reduces token consumption by 50.8%, and reaches 93.75% R@1 for GUI-state retrieval and 82.74% Top-1 accuracy for action re-grounding. These results show that historical GUI experience can serve as transferable, directly executable memory for reducing repeated VLM reasoning.

References

[] Reyna Abhyankar, Qi Qi, and Yiying Zhang. 2025. Osworld-human: Benchmarking the efficiency of computer-use agents. *arXiv preprint arXiv:2506.16042* (2025).

[] Gongwei Chen, Xurui Zhou, Rui Shao, Yibo Lyu, Kaiwen Zhou, Shuai Wang, Wentao Li, Yinchuan Li, Zhongang Qi, and Liqiang Nie. 2025. Less is more: Empowering gui agent with context-aware simplification. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 5901–5911.

[] Filippos Christianos, Georgios Papoudakis, Thomas Coste, Jianye HAO, Jun Wang, and Kun Shao. 2025. Lightweight Neural App Control. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=BL4WBIfyrz

[] Gaole Dai, Shiqi Jiang, Ting Cao, Yuanchun Li, Yuqing Yang, Rui Tan, Mo Li, and Lili Qiu. 2025. Advancing mobile gui agents: A verifier-driven approach to practical deployment. *arXiv preprint arXiv:2503.15937* (2025).

[] Tommaso Fulcini, Riccardo Coppola, Marco Torchiano, and Luca Ardito. 2023. An Analysis of Widget Layout Attributes to Support Android GUI-Based Testing. In *Proceedings of the IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE, 117–125. doi:10.1109/ICSTW58534.2023.00033

[] Lorenzo Gomez, Iulian Neamtiu, Tanzirul Azim, and Todd Millstein. 2013. Reran: Timing-and touch-sensitive record and replay for android. In *2013 35th international conference on software engineering (ICSE)*. IEEE, 72–81.

[] Han Hu, Haolan Zhan, Yujin Huang, and Di Liu. 2023. Pairwise GUI Dataset Construction Between Android Phones and Tablets. In *Advances in Neural Information Processing Systems*, Vol. 36. https://proceedings.neurips.cc/paper_files/paper/2023/file/bc4cff0b37ccab13e98b6128d89ca172-Paper-Datasets_and_Benchmarks.pdf

[] Ruihua Ji, Tingwei Zhu, Xiaoqing Zhu, Chunyang Chen, Minxue Pan, and Tian Zhang. 2023. Vision-Based Widget Mapping for Test Migration Across Mobile Platforms: Are We There Yet?. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 1416–1428. doi:10.1109/ASE56229.2023.00068

[] Jungjae Lee, Dongjae Lee, Chihun Choi, Youngmin Im, Jaeyoung Wi, Kihong Heo, Sangeun Oh, Sunjae Lee, and Insik Shin. 2025. VeriSafe Agent: Safeguarding Mobile GUI Agent via Logic-based Action Verification. In *Proceedings of the 31st Annual International Conference on Mobile Computing and Networking*. 817–831. doi:10.1145/3680207.3765248

[] Sunjae Lee, Junyoung Choi, Jungjae Lee, Munim Hasan Wasi, Hojun Choi, Steve Ko, Sangeun Oh, and Insik Shin. 2024. Mobilegpt: Augmenting llm with human-like app memory for mobile task automation. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*. 1119–1133.

[] Cong Li, Yanyan Jiang, and Chang Xu. 2022. Cross-Device Record and Replay for Android Apps. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 395–407. doi:10.1145/3540250.3549083

[] Toby Jia-Jun Li, Amos Azaria, and Brad A Myers. 2017. SUGILITE: creating multimodal smartphone automation by demonstration. In *Proceedings of the 2017 CHI conference on human factors in computing systems*. 6038–6049.

[] Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Shiwei Wu, Zechen Bai, Stan Weixian Lei, Lijuan Wang, and Mike Zheng Shou. 2025. Showui: One vision-language-action model for gui visual agent. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 19498–19508.

[] Philipp Lindenberger, Paul-Edouard Sarlin, and Marc Pollefeys. 2023. Lightglue: Local feature matching at light speed. In *2023 IEEE/CVF international conference on computer vision (ICCV)*. IEEE, 17581–17592.

[] Zibin Liu, Cheng Zhang, Xi Zhao, Yunfei Feng, Bingyu Bai, Dahu Feng, Erhu Feng, Yubin Xia, and Haibo Chen. 2025. Beyond Training: Enabling Self-Evolution of Agents with MOBIMEM. *arXiv preprint arXiv:2512.15784* (2025).

[] Maestro. 2026. Maestro. End-to-End UI Testing for Mobile and Web. https://maestro.dev/.

[] openatx. 2026. uiautomator2: Android Uiautomator2 Python Wrapper. https://github.com/openatx/uiautomator2.

[] Antti Oulasvirta, Tye Rattenbury, Lingyi Ma, and Eeva Raita. 2012. Habits make smartphone use more pervasive. *Personal and Ubiquitous computing* 16, 1 (2012), 105–114.

[] Mingyu Ouyang, Kevin Qinghong Lin, Mike Zheng Shou, and Hwee Tou Ng. 2026. FocusUI: Efficient UI Grounding via Position-Preserving Visual Token Selection. *arXiv preprint arXiv:2601.03928* (2026).

[] Qwen Team. 2026. Qwen3.6-Plus: Towards Real World Agents. https://qwen.ai/blog?id=qwen3.6

[] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William E Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Kenji Toyama, Robert James Berry, Divya Tyamagundlu, Timothy P Lillicrap, and Oriana Riva. 2025. AndroidWorld: A Dynamic Benchmarking Environment for Autonomous Agents. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=il5yUQsrjC

[] Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. 2023. Android in the wild: A large-scale dataset for android device control. *arXiv preprint arXiv:2307.10088* (2023).

[] Saghar Talebipour, Yixue Zhao, Luka Dojcilović, Chenggang Li, and Nenad Medvidović. 2021. UI Test Migration Across Mobile Platforms. In *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 756–767. doi:10.1109/ASE51524.2021.9678643

[] Shizuo Tian, Hao Wen, Yuxuan Chen, Jiacheng Liu, Shanhui Zhao, Guohong Liu, Ju Ren, Yunxin Liu, and Yuanchun Li. 2025. AgentProg: Empowering Long-Horizon GUI Agents with Program-Guided Context Management. *arXiv preprint arXiv:2512.10371* (2025).

[] Bryan Wang, Gang Li, and Yang Li. 2023. Enabling conversational interaction with mobile ui using large language models. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. 1–17.

[] Yuan Wang, Mingyu Li, and Haibo Chen. 2026. From Imperative to Declarative: Towards LLM-friendly OS Interfaces for Boosted Computer-Use Agents. In *Proceedings of the 21st European Conference on Computer Systems*. 296–310.

[] Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. 2024. Autodroid: Llm-powered task automation in android. In *Proceedings of the 30th annual international conference on Mobile computing and networking*. 543–557.

[] Hao Wen, Shizuo Tian, Borislav Pavlov, Wenjie Du, Yixuan Li, Ge Chang, Shanhui Zhao, Jiacheng Liu, Yunxin Liu, Ya-Qin Zhang, et al. 2025. Autodroid-v2: Boosting slm-based gui agents via code generation. In *Proceedings of the 23rd Annual International Conference on Mobile Systems, Applications and Services*. 223–235.

[] Mulong Xie, Jiaming Ye, Zhenchang Xing, and Lei Ma. 2023. NiCro: Purely Vision-based, Non-intrusive Cross-Device and Cross-Platform GUI Testing. *CoRR* abs/2305.14611 (2023). arXiv:2305.14611 doi:10.48550/arXiv.2305.14611

[] Tingxin Yan, David Chu, Deepak Ganesan, Aman Kansal, and Jie Liu. 2012. Fast app launching for mobile devices using predictive user context. In *Proceedings of the 10th international conference on Mobile systems, applications, and services*. 113–126.

[] Juyeon Yoon, Seungjoon Chung, Kihyuck Shin, Jinhan Kim, Shin Hong, and Shin Yoo. 2022. Repairing Fragile GUI Test Cases Using Word and

Layout Embedding. In *Proceedings of the IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 291–301. doi:10.1109/ICST53961.2022.00038

[] Chaoyun Zhang, Shilin He, Jiaxu Qian, Bowen Li, Liqun Li, Si Qin, Yu Kang, Minghua Ma, Guyue Liu, Qingwei Lin, et al. 2024. Large Language Model-Brained GUI Agents: A Survey. *arXiv preprint arXiv:2411.18279* (2024).

[] Chaoyun Zhang, He Huang, Chiming Ni, Jian Mu, Si Qin, Shilin He, Lu Wang, Fangkai Yang, Pu Zhao, Chao Du, et al. 2025. Ufo2: The desktop agentos. *arXiv preprint arXiv:2504.14603* (2025).

[] Chi Zhang, Zhao Yang, Jiaxuan Liu, Yanda Li, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. 2025. AppAgent: Multimodal Agents as Smartphone Users. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. 1–20.

[] Li Zhang, Longxi Gao, and Mengwei Xu. 2026. Does Chain-of-Thought Reasoning Help Mobile GUI Agents? An Empirical Study. In *Findings of the Association for Computational Linguistics: ACL 2026*. 7981–7996.

[] Hongbin Zhong, Fazle Faisal, Luis França, Tanakorn Leesataporn-wongsa, Adriana Szekeres, Kexin Rong, and Suman Nath. 2026. Actionengine: From reactive to programmatic gui agents via state machine memory. *arXiv preprint arXiv:2602.20502* (2026). https://arxiv.org/abs/2602.20502